K-Means Clustering

Beginner-friendly explanation with dataset, sklearn code, random_state, elbow method, and silhouette score

Goal: Learn how K-Means groups similar data points without labels, and how to run it using Python and scikit-learn.

Concept	Meaning
ML type	Unsupervised learning
Algorithm	K-Means Clustering
Input	Features only, no target label
Output	Cluster number for each row
Main parameter	n_clusters
Common use	Customer segmentation, grouping similar items

1. What is clustering?

Clustering is an unsupervised machine learning task. It groups data points that look similar to each other. Unlike classification, clustering does not need a known label such as Pass/Fail, Spam/Not spam, or Churn/Not churn.

In classification, the model learns from examples that already have answers. In clustering, the model tries to discover natural groups by looking at the feature values.

Task	Question	Example Algorithm	Example Output
Regression	Predict a number	Linear Regression	Marks = 85
Classification	Predict a known class	Logistic Regression Classifier	Spam / Not spam
Clustering	Find hidden groups	K-Means Clustering	Cluster 0 / Cluster 1 / Cluster 2

Simple memory: Classification predicts existing labels. Clustering creates groups when labels are not given.

2. What is K-Means?

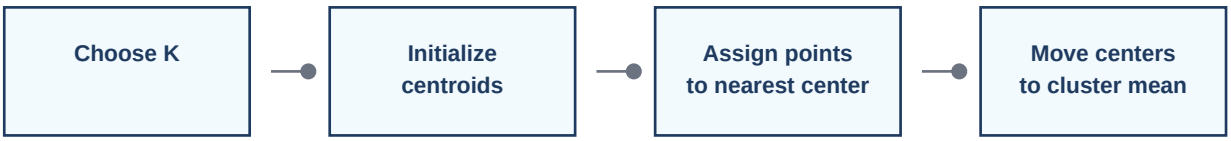
K-Means is a clustering algorithm that separates data into **K groups**. Each group has a center point called a **centroid**. Every data point is assigned to the nearest centroid.

The official scikit-learn clustering guide describes KMeans as separating samples into n groups of equal variance while minimizing inertia, also called within-cluster sum of squares. The number of clusters must be specified before training.

The name K-Means has two parts:

- **K** means the number of clusters you want.
- **Means** means the algorithm uses the mean/average position of points as the cluster center.

Example: KMeans(n_clusters=3) means the algorithm will create 3 groups.



Repeat assignment and center-moving until centers stop moving significantly

3. Sample dataset: customer grouping

Suppose we have customer data. We do not know customer types in advance, but we want to group similar customers using two features: annual income and spending score.

Customer	Annual_Income_k	Spending_Score
C1	15	20
C2	18	25
C3	22	23
C4	24	27
C5	35	50
C6	38	55
C7	60	70
C8	65	75
C9	82	88
C10	90	92

Here there is no target column like Churn or Approved. That is why this is clustering, not classification.

The scatter plot helps us visually see that some customers are close to each other. K-Means will turn this visual similarity into cluster labels.

4. K-Means sklearn code - basic example

This is the simplest version using the 10-row customer dataset.

```
import pandas as pd
from sklearn.cluster import KMeans

# 1. Create dataset
data = {
    "Customer": ["C1", "C2", "C3", "C4", "C5", "C6", "C7", "C8", "C9", "C10"],
    "Annual_Income_k": [15, 18, 22, 24, 35, 38, 60, 65, 82, 90],
    "Spending_Score": [20, 25, 23, 27, 50, 55, 70, 75, 88, 92]
}

df = pd.DataFrame(data)

# 2. Select features only. No y target is needed.
X = df[["Annual_Income_k", "Spending_Score"]]

# 3. Create K-Means model
model = KMeans(n_clusters=3, random_state=42, n_init="auto")

# 4. Train and get cluster labels
clusters = model.fit_predict(X)

# 5. Add cluster result to original table
df["Cluster"] = clusters

print(df)
print("Cluster centers:")
print(model.cluster_centers_)
print("Inertia:", model.inertia_)
```

Important: K-Means uses only X. There is no y because no correct answer label is provided.

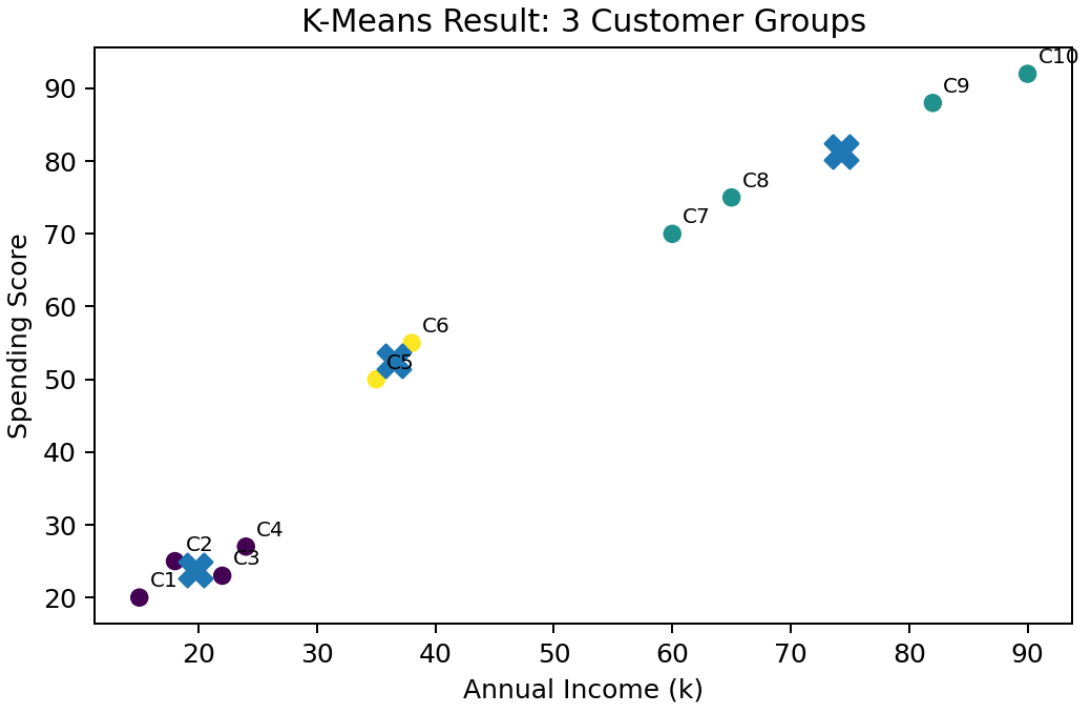
Output meaning: Each row gets a cluster label such as 0, 1, or 2. These numbers are just group IDs. Cluster 0 does not mean better than Cluster 1.

Customer	Annual_Income_k	Spending_Score	Cluster
C1	15	20	0
C2	18	25	0
C3	22	23	0
C4	24	27	0
C5	35	50	2
C6	38	55	2
C7	60	70	1
C8	65	75	1
C9	82	88	1
C10	90	92	1

5. Understanding the result

After fitting K-Means, we mainly look at three things:

- **labels_**: the cluster number assigned to each row.
- **cluster_centers_**: the center point of each cluster.
- **inertia_**: total squared distance between points and their assigned centers. Lower inertia means points are closer to their centers, but it always decreases as K increases.



The X markers show centroids. Customers near the same centroid belong to the same cluster.

A business interpretation could be:

- Low income and low spending customers
- Medium income and medium spending customers
- High income and high spending customers

> The exact cluster number may be different on another run or another dataset. Interpret clusters by looking at feature averages, not by trusting the numeric label name.

6. What does random_state do in K-Means?

K-Means starts by choosing initial centroid positions. This start can involve randomness. random_state=42 fixes that randomness so you get the same result when you run the same code on the same data.

Code part	Meaning
n_clusters=3	Create 3 groups
random_state=42	Use seed 42 so initialization is reproducible
n_init="auto"	Let sklearn choose number of initial runs based on init method

Very important: random_state=42 does not create 42 groups. The number of groups comes from n_clusters.

```
# Same cluster count, different seeds
KMeans(n_clusters=3, random_state=0, n_init="auto")
KMeans(n_clusters=3, random_state=42, n_init="auto")
KMeans(n_clusters=3, random_state=100, n_init="auto")

# All create 3 clusters because n_clusters=3.
# The random_state only controls the random starting point.
```

In official sklearn documentation, random_state determines random number generation for centroid initialization, and using an integer makes the randomness deterministic.

7. How to choose n_clusters?

K-Means needs you to choose n_clusters. There is no single perfect rule, but two common beginner-friendly methods are the elbow method and silhouette score.

A. Elbow method

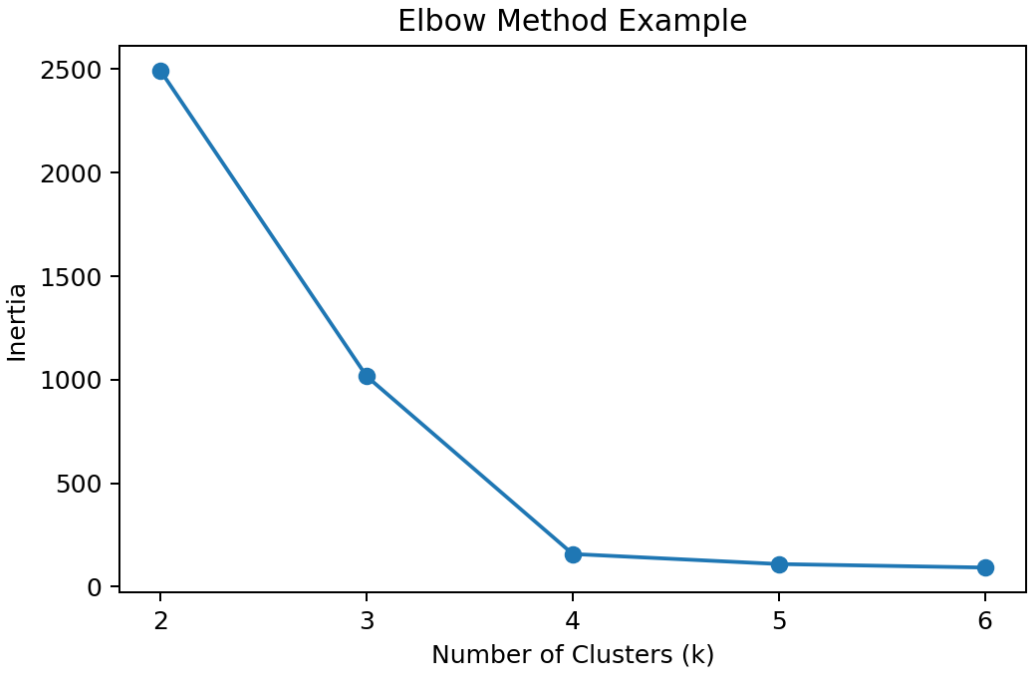
Run K-Means for different K values and store inertia. Inertia usually decreases as K increases. Look for the point where the decrease slows down. That bend is called the elbow.

```
from sklearn.cluster import KMeans

inertias = []
K_values = range(1, 8)

for k in K_values:
    model = KMeans(n_clusters=k, random_state=42, n_init="auto")
    model.fit(X)
    inertias.append(model.inertia_)

print(inertias)
```



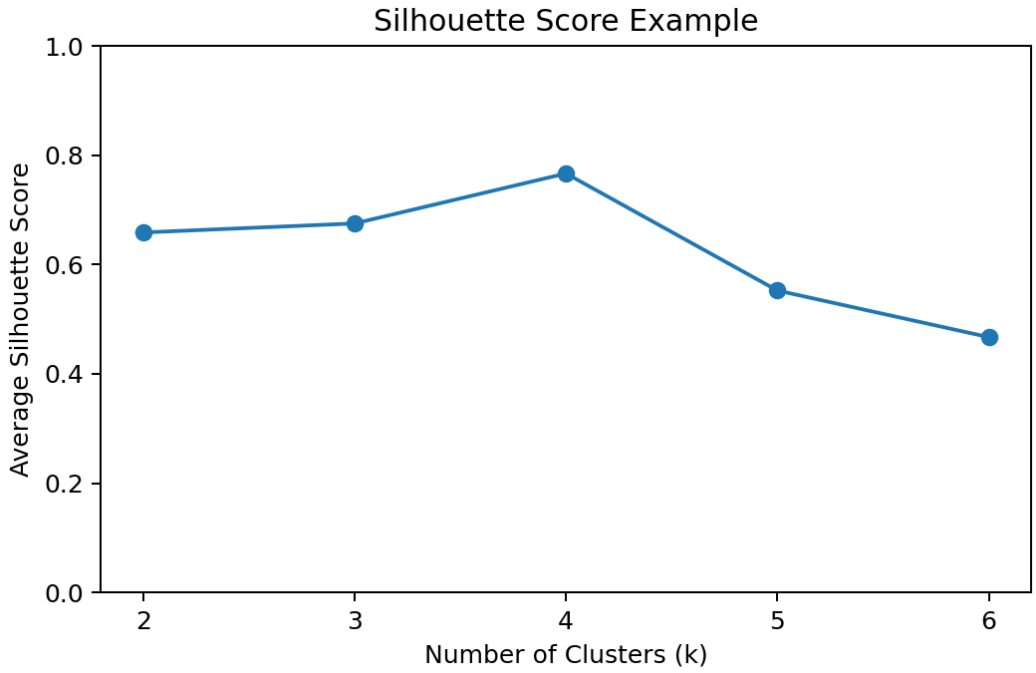
The elbow method is useful, but sometimes the elbow is not very clear.

B. Silhouette score

Silhouette score checks how well each point fits inside its cluster compared with nearby clusters. Scores are usually between -1 and +1. Higher is generally better. A score near +1 means the point is well placed inside its cluster. A score near 0 means it is close to a boundary. A negative value can mean the point may be assigned to the wrong cluster.

```
from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans

for k in range(2, 7):
    model = KMeans(n_clusters=k, random_state=42, n_init="auto")
    labels = model.fit_predict(X)
    score = silhouette_score(X, labels)
    print(k, score)
```



8. Why scaling is important

K-Means uses distance. If one feature has a very large scale, it can dominate the distance calculation.

Example: If income is measured in thousands and age is measured in years, income may have a bigger numeric range. K-Means might focus more on income than age. To avoid this, scale features before clustering.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

features = df[["Annual_Income_k", "Spending_Score"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(features)

model = KMeans(n_clusters=3, random_state=42, n_init="auto")
df["Cluster"] = model.fit_predict(X_scaled)

print(df)
```

Simple rule: For K-Means, scaling is usually recommended when feature units or ranges are different.

9. Predicting the cluster of a new customer

After training, you can assign a new data point to the nearest existing cluster center using predict.

```
new_customer = [[70, 80]]  # Annual income = 70k, spending score = 80
predicted_cluster = model.predict(new_customer)
print(predicted_cluster)
```

If you used scaling during training, you must scale the new customer using the same scaler before prediction.

```
new_customer = [[70, 80]]
new_customer_scaled = scaler.transform(new_customer)
predicted_cluster = model.predict(new_customer_scaled)
print(predicted_cluster)
```

> Do not fit a new scaler on the new customer. Use the already trained scaler from the training data.

10. Complete sklearn workflow

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Dataset
df = pd.DataFrame({
    "Customer": ["C1", "C2", "C3", "C4", "C5", "C6", "C7", "C8", "C9", "C10"],
    "Annual_Income_k": [15, 18, 22, 24, 35, 38, 60, 65, 82, 90],
    "Spending_Score": [20, 25, 23, 27, 50, 55, 70, 75, 88, 92]
})

# Select features
X = df[["Annual_Income_k", "Spending_Score"]]

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train K-Means
model = KMeans(n_clusters=3, random_state=42, n_init="auto")
df["Cluster"] = model.fit_predict(X_scaled)

# Evaluate cluster separation
score = silhouette_score(X_scaled, df["Cluster"])

print(df)
print("Cluster centers in scaled space:")
print(model.cluster_centers_)
print("Silhouette score:", score)
```

This is a good beginner project structure: load data, select features, scale features, fit K-Means, add cluster labels, then interpret clusters.

11. How to interpret clusters in real projects

K-Means only gives cluster numbers. The human/data analyst must give meaning to each cluster.

```
# View average feature values per cluster
cluster_summary = df.groupby("Cluster")[["Annual_Income_k", "Spending_Score"]].mean()
print(cluster_summary)

# View number of customers in each cluster
print(df["Cluster"].value_counts())
```

Example interpretation table:

Cluster pattern	Possible business name	Possible action
Low income + low spending	Budget customers	Discount offers
Medium income + medium spending	Regular customers	Personalized campaigns
High income + high spending	Premium customers	Loyalty benefits

Cluster names like Budget, Regular, and Premium are not created by K-Means. They are business labels added after analysis.

12. Common mistakes

Mistake	Correction
Using y in K-Means	K-Means is unsupervised, so use X only
Thinking cluster numbers have ranking	Cluster 2 is not automatically better than Cluster 0
Not scaling features	Scale when units/ranges differ
Choosing K blindly	Use elbow/silhouette/domain knowledge
Using K-Means for every shape	K-Means works best for compact, roughly round clusters

K-Means can give weak results when clusters are long, curved, heavily overlapping, or very different in size/density. In such cases, try DBSCAN, HDBSCAN, Gaussian Mixture Models, or hierarchical clustering.

13. K-Means mini project idea

Project title: Customer Segmentation using K-Means Clustering

Objective: Group customers based on income and spending behavior.

Steps:

- Collect or create customer data.
- Select numerical features.
- Scale features using StandardScaler.
- Use elbow method and silhouette score to choose K.
- Train KMeans.
- Add cluster labels to the dataset.
- Analyze average values per cluster.

- Give meaningful names to clusters.
- Write business recommendations.

Final result sentence:

I used K-Means Clustering to group customers into similar segments based on annual income and spending score. Since the data had no predefined labels, this was an unsupervised learning problem.

14. Quick revision

Question	Answer
What is clustering?	Grouping similar points without labels
What does K mean?	Number of clusters
What is centroid?	Center of a cluster
What does random_state do?	Makes random start reproducible
What is inertia?	Distance-based compactness measure
Why scaling?	Because K-Means uses distances
How to choose K?	Elbow method, silhouette score, domain knowledge

One-line summary:

K-Means is an unsupervised clustering algorithm that creates K groups by repeatedly assigning points to the nearest centroid and moving centroids to the average position of assigned points.

References

1. scikit-learn KMeans API documentation: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html

2. scikit-learn Clustering user guide, K-means section: https://scikit-learn.org/stable/modules/clustering.html#k-means

3. scikit-learn silhouette analysis example: https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html

Created for beginner learning and classroom/demo use.